

Lab 15 — Field Updates: OTA and the Rollback Safety Net

ESP32-S3 FIRMWARE LAB SERIES · PHASE 2 · PAST THE COURSEWORK

Estimated time	3 hrs
Introduces	A/B slots, image validation, automatic rollback
Prerequisites	Lab 13 (a working Wi-Fi connection).
Beyond the board	Wi-Fi, a host PC serving the image, and 16 MB of flash for two app slots.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **§15.1 objective**, **§15.2 parts and wiring**, **§15.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

15.1 Objective & TM4C bridge

Replace the running firmware over the network, and make it impossible for a bad update to brick the device. The second half is the part that matters; anyone can download a file.

TM4C BRIDGE

On the TM4C you reflashed over a cable, standing next to the board, and if the image was broken you simply flashed again. Neither of those is available to a device in the field, which is why the entire design below exists.

15.2 Parts & wiring

Item	Status	Notes
ESP32-S3 board with 16 MB flash	Required	The flash-size setting must be correct or the two app slots will not fit.
Wi-Fi, working from Lab 13	Required	—

Item	Status	Notes
A web server on your laptop	one command	<code>python -m http.server 8070</code> in the folder containing your new <code>.bin</code> .

WARNING**This is the lab where a 2 MB flash setting finally bites**

Two application slots of roughly 1.8 MB each cannot fit in a 2 MB configuration. If you have not already fixed it, do so now: `menuconfig` → Serial flasher config → Flash size → 16 MB. The symptom otherwise is a partition-table error that reads like image corruption.

15.3 How it works

OTA means Over-The-Air update: replacing the firmware on a device you cannot physically reach. The download is the easy part; not bricking the device is the engineering.

The requirement, stated precisely

Write down what must be true and the design follows from it: **at every single instant, including during a power failure mid-update, there must be a complete working firmware image on the device.**

That immediately rules out the obvious approach. You cannot overwrite the running program, because there is a window — from the first byte written until the last — where neither the old nor the new image is complete. Lose power in that window and the device is a brick with no way to recover except physical access, which is the one thing you do not have.

A/B slots

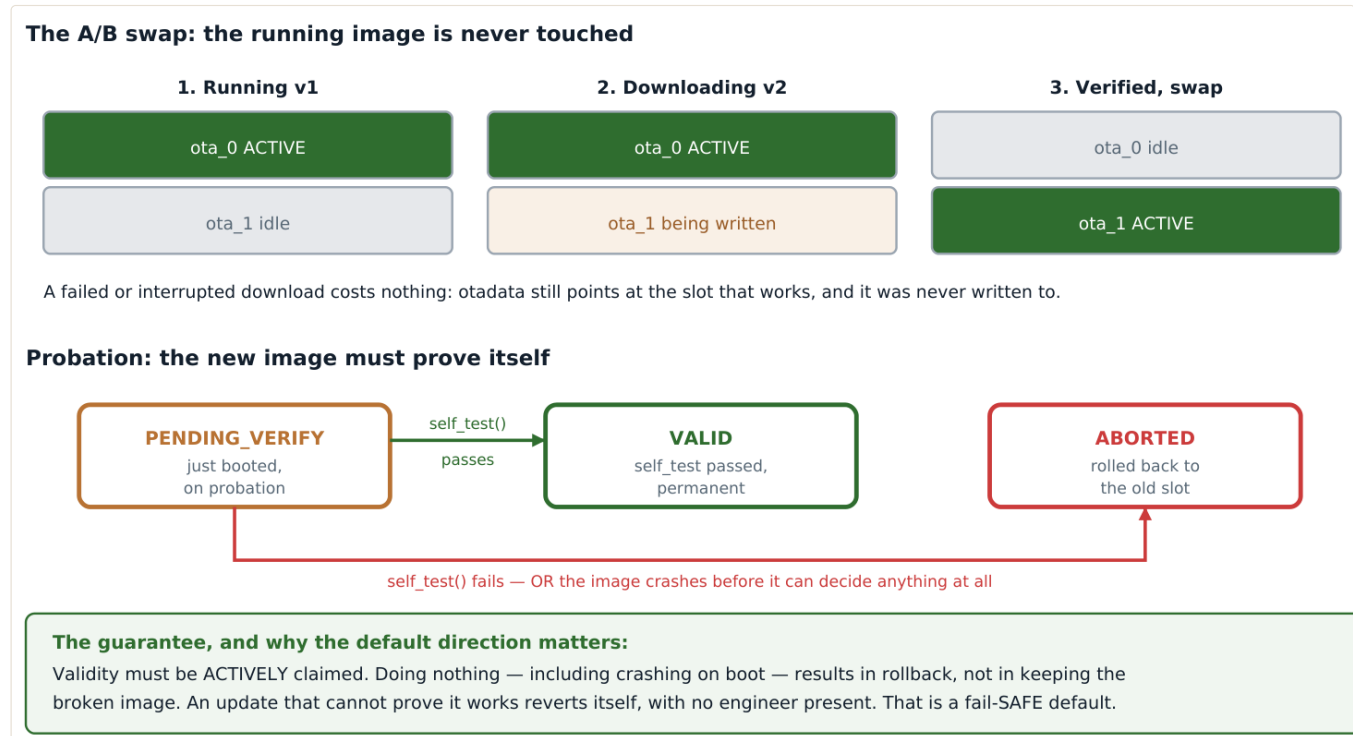


Figure 15.1. Top: the update is written entirely into the inactive slot while the running image is never touched, and only a tiny bookkeeping partition changes to swap them. Bottom: a freshly booted image must actively prove itself, and doing nothing results in rollback.

The A/B slot swap, and the probation state machine.

The swap, in three stages:

Stage	ota_0	ota_1
1. Running v1	ACTIVE	idle
2. Downloading v2	ACTIVE (untouched)	being written
3. Verified, swap	idle	ACTIVE

A failed or interrupted download costs nothing: `otadata` still points at the slot that works, and that slot was never written to.

Probation, the state machine a newly installed image goes through: - `PENDING_VERIFY` -- just booted, on probation. - On `self_test()` passing -> `VALID` -- permanent, the bootloader will never revert it. - On `self_test()` failing, OR if the image crashes before it can decide anything at all -> `ABORTED` -- rolled back to the old slot.

The guarantee, and why the default direction matters: validity must be ACTIVELY claimed. Doing nothing, including crashing on boot, results in rollback rather than in keeping the broken image. An update that cannot prove it works reverts itself, with no engineer present. That is a fail-SAFE default.

The flash carries **two complete application partitions**, conventionally `ota_0` and `ota_1`, each large enough for a full image. One is running; the other is spare. A third, tiny partition called `otadata` records which one the bootloader should launch.

An update writes entirely into the *inactive* slot while the current firmware keeps running untouched. Only when the new image is completely downloaded and its integrity verified does `otadata` get updated to point at it. So:

- A download that fails halfway costs nothing — the running image was never written to.
- A corrupted download is caught by the integrity check and never becomes active.
- A power cut mid-download leaves `otadata` still pointing at the working slot.

The cost is half your flash. Two 1.8 MB slots is 3.6 MB of a 16 MB part committed to holding two copies of the same program. That is exactly why the flash-size setting matters here and nowhere earlier.

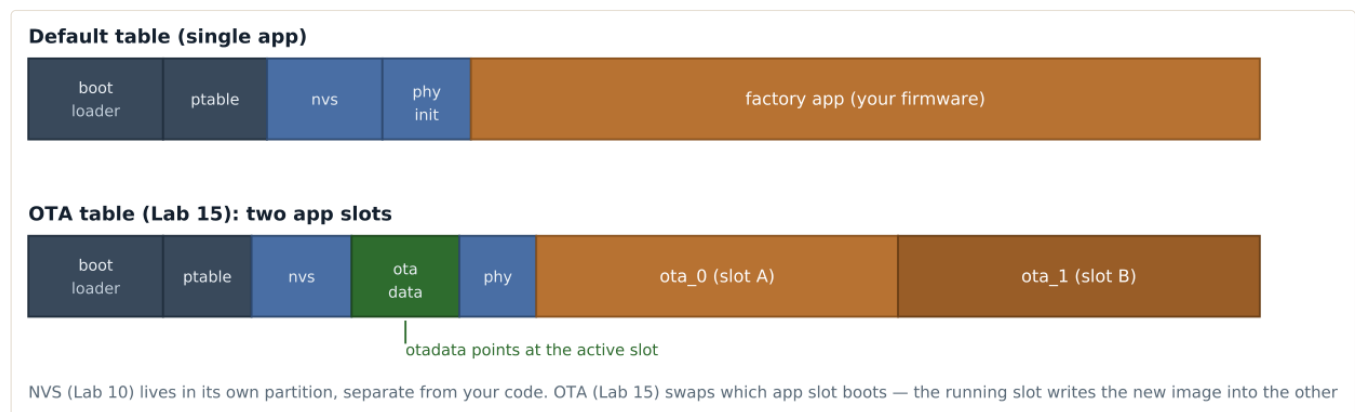


Figure 15.2. The same layout as a map of the flash. Note the proportions: two app slots dominate, while `otadata` and `nvs` are slivers — and it is the sliver that decides which slot boots.

Flash partition layouts, single-app versus OTA.

Default table (single app), left to right: bootloader | partition table | nvs | phy_init | factory app (your firmware, occupying most of the flash).

OTA table (Lab 15), left to right: bootloader | partition table | nvs | otadata | phy | ota_0 | ota_1. The two app slots are equal in size and together take most of the flash.

`otadata` points at the currently active slot. It is tiny compared to the app slots, yet it is what decides which one boots.

NVS (Lab 10) lives in its own partition, separate from your code, which is why stored settings survive a firmware update. OTA (Lab 15) swaps which app slot boots: the running slot writes the new image into the other one.

Probation: the failure A/B slots do not cover

A/B protects against a failed *download*. It does nothing about a *successful* download of firmware that is genuinely broken — an image that flashes perfectly, passes its checksum, boots, and then crashes,

or connects to nothing, or has a bug that stops it ever accepting another update. Without more machinery you have just remotely bricked the device with a perfectly valid file.

So a freshly installed image boots in a **probationary** state, `ESP_OTA_IMG_PENDING_VERIFY`. In that state the firmware must actively declare itself healthy by calling `esp_ota_mark_app_valid_cancel_rollback()`. If it decides it is broken, it calls `esp_ota_mark_app_invalid_rollback_and_reboot()` and the device reverts.

And if it crashes before doing either, the next boot finds the image still marked pending and the bootloader rolls back automatically. That is the complete guarantee, and the direction of the default is the whole trick: **validity must be actively claimed, and doing nothing means rollback**. An update that cannot prove it works reverts itself, with no engineer present.

KEY POINT

Your self_test() is the part that actually matters

The rollback machinery is only as good as the test that gates it. A `self_test()` that returns `true` unconditionally makes the whole mechanism decorative — a broken image will cheerfully declare itself healthy.

A meaningful one checks whatever "working" means for your device: does the IMU answer WHO_AM_I, does Wi-Fi associate, do sensor readings fall in a plausible range, can you still reach the update server. Writing that function is the real engineering in this lab, and it is what an interviewer will probe if you mention OTA.

Why the download must be authenticated

Firmware update is the most security-sensitive operation a device performs, because **whoever controls the firmware controls the device completely**. An attacker who can serve you a fake image is not stealing data; they own the hardware.

So the download runs over **TLS** — the same encryption behind HTTPS — with the server's certificate verified against a bundle of trusted certificate authorities. That proves you are talking to the server you think you are, and that nobody modified the bytes in transit.

Production goes further and **signs the image itself**, so the device verifies the author cryptographically, independently of how it arrived. Combined with **secure boot**, which makes the chip refuse to execute an unsigned image at all, that is what stops someone with physical access from simply flashing their own firmware. This lab uses plain HTTP on your own network for simplicity, and you should be clear-eyed that this is a lab convenience and not an acceptable product design.

One more production feature worth knowing by name: **anti-rollback**, which stops an attacker from "updating" you to a genuine but older image with a known vulnerability. It works by burning a monotonic version counter into eFuse so the chip refuses anything older.

How the pieces fit together

Piece	Job
Partition table	Declares the two app slots and <code>otadata</code> . Set at build time.
<code>esp_https_ota()</code>	Downloads, verifies and writes into the inactive slot, then updates <code>otadata</code> .

Piece	Job
Bootloader	Reads <code>otadata</code> on every boot and launches the indicated slot. Rolls back automatically if it finds an unverified image.
Your <code>confirm_or_rollback()</code>	Runs early on every boot, and if this image is on probation, decides its fate.

15.4 Procedure & code

1. Select an OTA-capable partition table in `menuconfig` → Partition Table → "Factory app, two OTA definitions."
2. Build, flash normally once over USB. This is the image that will perform the first update.
3. Change something visible (a log string), rebuild, and serve the new `.bin` from your laptop.
4. Trigger the update and watch it swap.

```
// ---- partition table (partitions.csv) ----
# Name,   Type, SubType, Offset,   Size
nvs,      data, nvs,      0x9000,   0x6000
otadata,  data, ota,      0xf000,   0x2000
phy_init, data, phy,      0x11000,  0x1000
ota_0,    app,  ota_0,    0x20000,  0x1C0000
ota_1,    app,  ota_1,    0x1E0000, 0x1C0000
```

```
// ---- performing the update ----
#include "esp_https_ota.h"
#include "esp_crt_bundle.h"
#include "esp_log.h"

static void do_ota(const char *url) {
    esp_http_client_config_t http = {
        .url = url,
        .crt_bundle_attach = esp_crt_bundle_attach, // verify the server over TLS
    };
    esp_https_ota_config_t ota = { .http_config = &http };

    if (esp_https_ota(&ota) == ESP_OK) {
        ESP_LOGI("ota", "update installed; rebooting into the new slot");
        esp_restart();
    } else {
        ESP_LOGE("ota", "update failed; staying on the current image");
    }
}
```

```
// ---- the safety net: run this early on every boot ----
#include "esp_ota_ops.h"

static void confirm_or_rollback(void) {
    const esp_partition_t *running = esp_ota_get_running_partition();
    esp_ota_img_states_t state;
    if (esp_ota_get_state_partition(running, &state) == ESP_OK &&
        state == ESP_OTA_IMG_PENDING_VERIFY) {
```

```

        if (self_test()) esp_ota_mark_app_valid_cancel_rollback(); // keep it
        else             esp_ota_mark_app_invalid_rollback_and_reboot(); // revert
    }
}

```

15.5 Code walkthrough, line by line

Three blocks: a configuration file, the download, and the safety net. The third is the one worth reading twice.

Block 1, line 2 · # Name, Type, SubType, Offset, Size

The partition table is a plain CSV file, which is refreshingly transparent — you can read the entire flash layout of your device in seven lines. It is compiled into a small binary blob and written near the start of flash, where the bootloader reads it.

Type is broadly `app` or `data`. **SubType** refines that: `ota_0`, `ota_1` and `factory` for app partitions; `nvs`, `ota`, `phy` for data.

Block 1, line 3 · nvs, data, nvs, 0x9000, 0x6000

Your Lab 10 key-value store: 24 KB starting at 36 KB into the flash. **Notice it sits outside both app slots**, and that is why your stored settings survive a firmware update — the update only rewrites an app partition, so it physically cannot touch this region. That separation is a deliberate design decision, not an accident of layout.

Block 1, line 4 · otadata, data, ota, 0xf000, 0x2000

The bookkeeping partition — only 8 KB, because all it records is which app slot is active and what state it is in. This tiny region is the entire swap mechanism: an update does not move any code, it changes *this*.

It is actually stored as two mirrored copies with sequence numbers, so that a power failure while updating `otadata` itself still leaves one valid copy. The safety reasoning goes all the way down.

Block 1, lines 6-7 · ota_0 and ota_1

The two application slots, and the important thing is what is **identical** about them: same type, same subtype family, and crucially **the same size**, `0x1C0000` — 1,835,008 bytes, about 1.8 MB each. That symmetry is what makes either slot able to hold a complete image, which is what makes them interchangeable.

The offsets differ: `0x200000` and `0x1E0000`. Check the arithmetic yourself — $0x200000 + 0x1C0000 = 0x1E0000$ — so `ota_1` starts exactly where `ota_0` ends, with no gap and no overlap. App partitions must also be aligned to a 64 KB boundary, which is why the first one starts at `0x200000` rather than immediately after `phy_init`.

Total: about 3.6 MB of app space plus overhead, which fits comfortably in 16 MB and not at all in 2 MB.

Block 2, lines 7-10 · `esp_http_client_config_t http = { .url = url, .cert_bundle_attach = ... };`

The same HTTP client configuration struct you met in Lab 13, reused here because an OTA download is an HTTP GET — just one whose response body happens to be a firmware image.

`.crt_bundle_attach = esp_crt_bundle_attach` is the security line. It attaches a bundle of trusted certificate-authority root certificates, compiled into your firmware, so the TLS layer can verify the server's certificate chain. Without it, you would either accept any certificate — which defeats the point — or have to pin one specific certificate yourself.

Two practical gotchas with TLS on an embedded device: the certificate bundle costs flash (tens of kilobytes), and validation checks certificate *expiry dates*, so a device whose clock is badly wrong will reject perfectly good certificates. That is a genuinely confusing failure the first time you meet it.

Block 2, line 11 · `esp_https_ota_config_t ota = { .http_config = &http };`

The OTA config wraps the HTTP config, which is a nice illustration of the layering: OTA is a *consumer* of the HTTP client rather than a replacement for it. The struct has other fields you would use in production — a callback fired after the image header is read, so you can inspect the incoming version and refuse a downgrade before downloading megabytes.

Block 2, line 13 · `if (esp_https_ota(&ota) == ESP_OK)`

One call doing a great deal, and it is worth knowing what happens inside it because the details are what make it safe.

It opens the TLS connection, finds the inactive partition, and **streams** the image into it as bytes arrive — never buffering the whole thing in RAM, which would be impossible for a 1.8 MB image on a chip with 512 KB. It verifies the image header and checksum. Only if everything checks out does it update `otadata` to point at the new slot.

It is **synchronous and slow** — tens of seconds over a typical connection — so it belongs in its own task, never in a callback. If anything fails at any stage, it returns an error and `otadata` is left untouched.

Block 2, lines 14-18 · the success and failure branches

The structure is the point. On success you `esp_restart()` into the new image. On failure you log and **carry on running the old firmware**.

Read that again: there is no error path that leaves the device in a bad state. Not because the code is careful about cleanup, but because the design makes a bad state unreachable — the running image was never modified. **Safety by construction beats safety by carefulness**, and that distinction is worth being able to articulate.

Block 3, line 5 · `const esp_partition_t *running = esp_ota_get_running_partition();`

Asks which partition is currently executing. **The firmware genuinely does not know this at compile time**, because the identical binary can be flashed into either slot — a fact that is slightly disorienting the first time you think about it. The same bytes run from two different addresses depending on which slot the bootloader chose.

Returns a pointer to a descriptor the system owns; you read it, you do not free it.

Block 3, lines 6-8 · `esp_ota_get_state_partition(running, &state) == ESP_OK && state == ESP_OTA_IMG_PENDING_VERIFY`

Reads that partition's recorded state into `state` via an output parameter, then checks for probation.

Two things worth noting. The **&& short-circuits**: if the first call fails, C never evaluates the second condition, so `state` is not read while uninitialised. That ordering is deliberate.

And the whole block does nothing at all on a normal boot — if the image is already `VALID`, there is no probation to resolve. So this function is safe to call unconditionally at the top of every boot, which is exactly how you should use it.

Block 3, line 9 · `if (self_test()) esp_ota_mark_app_valid_cancel_rollback();`

`self_test()` is **your** function, and §15.3 argues it is the most important part of the lab. The call it gates makes the image permanent: the pending state is cleared and the bootloader will never revert it.

The function name is unusually explicit — "mark valid" *and* "cancel rollback" — because it does both, and the naming reflects that skipping it is not a neutral omission.

Block 3, line 10 · `else esp_ota_mark_app_invalid_rollback_and_reboot();`

Discards this image and reboots into the previous slot. Note it **does not return** — the reboot is part of the call.

Look at the asymmetry in the two function names one more time, because it encodes the safety property from §15.3: validity must be *actively claimed*, while invalidity is what you get by default. If this function is never reached — because the image crashed, or hung, or the watchdog fired first — the image stays marked pending and the next boot rolls it back anyway. **The failure path requires no cooperation from the broken firmware**, which is precisely what makes it trustworthy.

15.6 Docs & references

DOCS

Trace every line to its source

`esp_https_ota` — API Reference → System → **ESP HTTPS OTA**. Read about the advanced API too, which lets you drive the download in steps and report progress.

`esp_ota_get_running_partition`, `esp_ota_get_state_partition`, `esp_ota_mark_app_valid_cancel_rollback` — System → **OTA API**. The state diagram on that page is worth studying; it shows every transition an image can make.

Rollback and anti-rollback — the same page. Anti-rollback prevents downgrading to a known-vulnerable version, which is a real security feature worth knowing exists.

Partition tables — API Guides → **Partition Tables**, for the CSV format and the built-in options.

Secure boot and flash encryption — API Guides → Security. You will not implement these here, but knowing what they defend against is exactly the kind of thing a defense or automotive interviewer will appreciate.

15.7 Verify, troubleshoot, your turn

Verify. The board downloads, reboots, and your changed log string appears — proving new firmware arrived without a cable. Then check which slot is running; it should have swapped. Run it again and it

should swap back. Watching the device alternate between `ota_0` and `ota_1` across updates is the clearest demonstration that the A/B scheme is real.

Troubleshoot

Symptom	Cause and fix
Partition table errors at build	Flash size set to 2 MB. Two 1.8 MB slots do not fit.
Download starts then fails	Wrong URL, firewall, or the server is not serving the <code>.bin</code> .
Reboots into the same old image	<code>otadata</code> was not updated, meaning validation failed. Check the log for the reason.
Boot loops between the two slots	Your <code>self_test()</code> is failing, so each new image rolls itself back. That is the mechanism <i>working</i> ; fix the test or the firmware.
TLS errors	Certificate bundle not attached, or the device clock is wrong — certificate validation depends on the date.

Your turn

1. **Break an update on purpose.** Write a `self_test()` that returns false, push that image, and watch the device roll itself back to the working one with no help from you. This is the single most reassuring thing in the manual to have witnessed, and it makes a genuinely memorable interview story.
2. **Interrupt a download** by killing the server halfway. Confirm the device carries on running the old firmware. Then confirm it can still update afterwards.
3. **Write a real self-test** for your own project: check the sensor identifies itself, check a feature value is plausible, check connectivity. Then articulate what it does *not* cover — knowing the limits of your own safety net is the mature version of this skill.

Errata and corrections

Defects confirmed against the hardware and the vendor documentation, with the evidence that established each one. They are printed rather than quietly patched because how an error was caught is usually more instructive than the error.

rollback is never actually armed

WARNING

Confirmed defect. Severity: the lab's central lesson silently does nothing.

The lab teaches the probation state machine and has you call

`esp_ota_mark_app_valid_cancel_rollback()`. But **those states are not used unless a Kconfig option is enabled, and it is disabled by default.**

Evidence — Espressif's OTA documentation, present in identical wording from v4.1 through v6.0: if `CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE` is not enabled, *which is the default*, then

`esp_ota_mark_app_valid_cancel_rollback()` and `esp_ota_mark_app_invalid_rollback_and_reboot()` are optional, and the `ESP_OTA_IMG_NEW` and `ESP_OTA_IMG_PENDING_VERIFY` states are not used at all.

What you would have seen: `confirm_or_rollback()` runs, finds the state is not `PENDING_VERIFY`, does nothing, and returns. The update works, so it looks correct — but a deliberately broken image would **not** roll back. The safety property the lab claims to demonstrate is absent.

WARNING

Second confirmed defect in the same lab: HTTP versus HTTPS

§15.4 tells you to serve the image with `python -m http.server 8070`, which is plain HTTP. The listing calls `esp_https_ota()` and attaches `esp_crt_bundle_attach`, which exist to verify a TLS certificate.

§15.3 then says the lab "uses plain HTTP on your own network for simplicity." **Those three statements cannot all be true.** The procedure and the code disagree.

Fix: Part B below rebuilds this as a real HTTPS OTA with rollback genuinely enabled, and adds the four recorded test cases that prove it: valid image boots, self-test pass makes it permanent, self-test fail rolls back, and reset during probation rolls back.

Bench realities

Problems that stop this lab but are not about the peripheral it teaches. Each one cost real time on a real bench.

Calling `example_connect()` twice aborts the boot

WARNING

Symptom: `assert failed ... duplicate key in esp_netif_new_api, immediately at startup.`

Cause: `example_connect()` registers a network interface with a unique key. Call it a second time — easy to do when you paste the setup block into a task as well as into `app_main` — and the second registration collides.

Fix: call it exactly once, from `app_main`, before anything that needs the network. The error message names the real cause, which is unusually helpful; the difficulty is only that "duplicate key" does not obviously mean "you connected twice."

The flash-size setting, one more time

This has now caused problems in Labs 10, 15 and 23 and it is worth doing once, permanently: `idf.py menuconfig` → Serial flasher config → Flash size → **16 MB**. Then confirm rather than assume:

```
grep ESPTOOLPY_FLASHSIZE build/config/sdkconfig.h
```

Better still, put it in `sdkconfig.defaults` at the project root so every new project inherits it:

```
CONFIG_ESPTOOLPY_FLASHSIZE_16MB=y
CONFIG_ESPTOOLPY_FLASHSIZE="16MB"
CONFIG_SPIRAM=y
CONFIG_SPIRAM_MODE_OCT=y
```

Part B — OTA With Rollback Actually Enabled

Replaces: Lab 15 · **Fixes:** rollback never armed; HTTP/HTTPS contradiction · **Adds:** four recorded proof cases

15B.1 What was wrong

Two defects, and the first is the serious one.

Rollback was never armed. Lab 15 teaches the probation state machine and calls `esp_ota_mark_app_valid_cancel_rollback()`. But Espressif's documentation is explicit, in identical wording from v4.1 through v6.0: if `CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE` is not enabled — **and it is disabled by default** — then those two functions are optional and the `ESP_OTA_IMG_NEW` and `ESP_OTA_IMG_PENDING_VERIFY` states are **not used at all**.

So `confirm_or_rollback()` would run, find a state that is not `PENDING_VERIFY`, and quietly do nothing. The update still worked, which is what makes this nasty: the lab looks successful while the safety property it exists to teach is entirely absent.

HTTP versus HTTPS. §15.4 says to serve with `python -m http.server 8070` (plain HTTP), the code calls `esp_https_ota()` and attaches a certificate bundle (TLS), and §15.3 says the lab uses plain HTTP for simplicity. Three statements, mutually inconsistent.

15B.2 Configuration — do this first, it is the whole fix

```
idf.py menuconfig

  Bootloader config
    [*] Enable app rollback support          <-- CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE
        [ ] Enable app anti-rollback support (optional; burns an eFuse counter)

  Partition Table
    (X) Custom partition table CSV           (the two-slot table from 15.4)

  Component config -> ESP HTTPS OTA
    [*] Allow HTTP for OTA                  <-- ONLY for path A below
```

KEY POINT

Confirm it took effect

After building, check the generated config rather than trusting the menu:

```
text grep BOOTLOADER_APP_ROLLBACK build/config/sdkconfig.h #define
CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE 1
```

If that line is absent, everything below is decorative. This one grep is the difference between a lab that teaches rollback and a lab that pretends to.

15B.3 Pick one transport and say so

	Path A — local HTTP	Path B — real HTTPS
Server	<code>python -m http.server 8070</code> (GET only is fine; OTA is a GET)	<code>openssl</code> -generated dev CA + <code>http.server</code> wrapped in TLS
Firmware	<code>esp_https_ota</code> with no cert bundle, plus "Allow HTTP for OTA"	<code>esp_https_ota</code> with <code>.crt_bundle_attach</code> or a pinned CA
Teaches	the A/B and rollback mechanics in isolation	the same, plus server authentication
Honest label	not production	production-shaped

Do **Path A first** and get the four proof cases in §15B.6 passing, because debugging rollback and TLS at the same time is miserable. Then repeat as Path B. What you must not do is what Lab 15 did: mix the two and describe a third thing.

Note that OTA downloads an image with a **GET**, so unlike Lab 13 the stock `python -m http.server` is genuinely adequate for Path A.

15B.4 A `self_test()` that means something — every helper defined

WARNING

This section previously referenced functions it never defined

An earlier edition called `health_check_ok()`, `mpu_read()` and `i2c_bringup()` without supplying any of them, and pointed at a `/health` endpoint that no server in the manual served. **That is the exact failure this rebuild exists to prevent.** Everything below is complete.

The rollback machinery is only as good as the test gating it. A `self_test()` that returns `true` unconditionally makes the whole mechanism decorative.

15B.4.1 — The host: an OTA server that also answers `/health`

`python -m http.server` serves files but has no `/health` resource, so the DUT's reachability probe would always 404. Save this as `ota_server.py`. It needs nothing beyond the standard library.

```
#!/usr/bin/env python3
"""Lab 15B host: serves the firmware image AND a /health endpoint.

    python ota_server.py 8070 build/lab15b.bin

    GET /firmware.bin -> the image (what esp_https_ota fetches)
    GET /health       -> {"ok":true} (what the DUT's self-test probes)
"""
import sys, json, pathlib
from http.server import BaseHTTPRequestHandler, HTTPServer

IMAGE = pathlib.Path("firmware.bin")

class H(BaseHTTPRequestHandler):
```

```

def _send(self, code, body, ctype):
    self.send_response(code)
    self.send_header("Content-Type", ctype)
    self.send_header("Content-Length", str(len(body)))
    self.end_headers()
    self.wfile.write(body)

def do_GET(self):
    if self.path.startswith("/health"):
        self._send(200, json.dumps({"ok": True}).encode(), "application/json")
    elif self.path.startswith("/firmware.bin"):
        if not IMAGE.exists():
            self._send(404, b"no image", "text/plain"); return
        data = IMAGE.read_bytes()
        print(f"  serving {len(data):,} bytes to {self.client_address[0]}", flush=True)
        self._send(200, data, "application/octet-stream")
    else:
        self._send(404, b"not found", "text/plain")

def log_message(self, *a): pass

if __name__ == "__main__":
    port = int(sys.argv[1]) if len(sys.argv) > 1 else 8070
    if len(sys.argv) > 2:
        IMAGE = pathlib.Path(sys.argv[2])
    print(f"OTA server on:{port}  image={IMAGE}  ({'found' if IMAGE.exists() else 'MISSING'})")
    print(f"  GET /firmware.bin  GET /health")
    HTTPServer(("0.0.0.0", port), H).serve_forever()

```

```
python ota_server.py 8070 build\lab15b.bin
```

KEY POINT

Verified behaviour

```

text $ curl http://127.0.0.1:8070/health {"ok": true} <- HTTP 200
$ curl -o /dev/null http://127.0.0.1:8070/firmware.bin 4096 bytes <- HTTP 200
$ curl -o /dev/null http://127.0.0.1:8070/nonsense <- HTTP 404

```

15B.4.2 — The device side, complete

```

#include <string.h>
#include "esp_system.h"
#include "esp_ota_ops.h"
#include "esp_http_client.h"
#include "esp_wifi.h"
#include "driver/i2c_master.h"
#include "nvs_flash.h"
#include "nvs.h"
#include "esp_log.h"

static const char *TAG = "lab15b";
static i2c_master_dev_handle_t s_imu;

// ----- I2C
// Defined here rather than assumed. Same bring-up as Lab 5.

```

```

static esp_err_t i2c_bringup(void)
{
    i2c_master_bus_config_t bc = {
        .i2c_port = I2C_NUM_0, .sda_io_num = GPIO_NUM_8, .scl_io_num = GPIO_NUM_9,
        .clk_source = I2C_CLK_SRC_DEFAULT, .glitch_ignore_cnt = 7,
        .flags.enable_internal_pullup = true,
    };
    i2c_master_bus_handle_t bus;
    esp_err_t e = i2c_new_master_bus(&bc, &bus);
    if (e != ESP_OK) return e;

    i2c_device_config_t dc = {
        .dev_addr_length = I2C_ADDR_BIT_LEN_7,
        .device_address = 0x68,
        .scl_speed_hz = 400000,
    };
    return i2c_master_bus_add_device(bus, &dc, &s_imu);
}

static esp_err_t mpu_read(uint8_t reg, uint8_t *dst, size_t n)
{
    return i2c_master_transmit_receive(s_imu, &reg, 1, dst, n, pdMS_TO_TICKS(100));
}

// ----- reachability probe
// Association is NOT reachability. This actually talks to the server that
// would deliver the NEXT update. Bounded, and retried a few times so a
// server that happens to be restarting does not roll back a good image.
static bool health_check_ok(const char *url, int attempts, int timeout_ms)
{
    for (int i = 0; i < attempts; i++) {
        esp_http_client_config_t cfg = {
            .url = url,
            .method = HTTP_METHOD_GET,
            .timeout_ms = timeout_ms,
        };
        esp_http_client_handle_t c = esp_http_client_init(&cfg);
        if (!c) return false;

        esp_err_t err = esp_http_client_perform(c);
        int status = (err == ESP_OK) ? esp_http_client_get_status_code(c) : -1;
        esp_http_client_cleanup(c); // on BOTH paths, or you leak

        if (err == ESP_OK && status >= 200 && status < 300) {
            ESP_LOGI(TAG, "health ok (HTTP %d, attempt %d)", status, i + 1);
            return true;
        }
        ESP_LOGW(TAG, "health attempt %d/%d failed (err=%s status=%d)",
            i + 1, attempts, esp_err_to_name(err), status);
        vTaskDelay(pdMS_TO_TICKS(3000)); // documented budget, see below
    }
    return false;
}

// ----- self-test
static bool self_test(void)
{
    // 1. storage: the NVS PARTITION must be usable.
    //

```

```

// Careful here. An earlier version failed the board if the "storage"
// namespace could not be opened -- but on a FRESH unit nothing has ever
// been written, so the namespace does not exist yet and nvs_open returns
// ESP_ERR_NVS_NOT_FOUND. That version rolled back a perfectly good image
// on its very first boot.
//
// "Nothing stored yet" and "storage is broken" are different answers.
nvs_handle_t h;
esp_err_t e = nvs_open("storage", NVS_READONLY, &h);
if (e == ESP_OK) {
    nvs_close(h); // namespace exists and opened
} else if (e == ESP_ERR_NVS_NOT_FOUND) {
    ESP_LOGI(TAG, "self-test: no 'storage' namespace yet (fresh unit) -- OK");
} else {
    ESP_LOGE(TAG, "self-test: NVS unusable: %s", esp_err_to_name(e));
    return false; // a REAL storage fault
}

// 2. sensor: does the IMU still identify itself?
uint8_t who = 0;
if (mpu_read(0x75, &who, 1) != ESP_OK) {
    ESP_LOGE(TAG, "self-test: IMU did not ACK");
    return false;
}
if (who != 0x70 && who != 0x68 && who != 0x71) {
    ESP_LOGE(TAG, "self-test: IMU WHO_AM_I = 0x%02X", who);
    return false;
}

// 3a. associated? necessary, but NOT sufficient
wifi_ap_record_t ap;
if (esp_wifi_sta_get_ap_info(&ap) != ESP_OK) {
    ESP_LOGE(TAG, "self-test: not associated");
    return false;
}

// 3b. can we actually REACH the update service?
// 3 attempts x 3 s timeout + 3 s backoff = about 18 s worst case.
// Write that budget down and defend it; "check once" is not defensible.
if (!health_check_ok("http://192.168.1.50:8070/health", 3, 3000)) {
    ESP_LOGE(TAG, "self-test: update server unreachable");
    return false;
}

ESP_LOGI(TAG, "self-test PASS (rssi %d)", ap.rssi);
return true;
}

```

WARNING**Association is not reachability**

`esp_wifi_sta_get_ap_info()` proves only that the radio is associated with an access point. A device can be perfectly associated and still unable to resolve DNS, route, or reach your server.

That distinction matters more here than almost anywhere else. An image that boots and reads its sensor but cannot reach the update service is the **worst** possible OTA outcome: the device is alive, looks healthy, marks itself valid, and **can never be updated again**. Espressif's own guidance suggests a successful connection to the OTA server as the checkpoint before cancelling rollback.

Figure 49. The decision an image makes on its first boot. The two classes of check exist because they fail for different reasons and deserve different responses.

Probation decision flow: what happens on the first boot of a new image. Not every failed check should roll back.

Flow: **boot after OTA** -> is `state == PENDING_VERIFY`? If **no**, this is a normal boot, not on probation, and the whole flow is skipped. If **yes**, run two classes of check.

	CLASS A - boot-critical, local	CLASS B - network reachability
Checks	NVS partition readable; IMU answers WHO_AM_I; memory available	<code>GET /health</code> on the update server
Budget	none	3 attempts, 3 s timeout, 3 s backoff (about 18 s)
On failure	ROLL BACK IMMEDIATELY - these cannot fix themselves	RETRY the budget, THEN roll back - a restarting server is not a bad image

If all checks pass -> `esp_ota_mark_app_valid_cancel_rollback()` -> the image is now permanent.

The asymmetry is the design. A missing sensor will not fix itself in thirty seconds, so waiting only delays the inevitable. A server that happens to be restarting is not evidence of a bad image, and rolling back over it is its own availability bug.

Note the branch out the right-hand side: **most boots are not probation boots**. After an image is marked valid once, this whole flow is skipped, so the machinery costs you once per update rather than on every start.

And: do nothing at all - including crashing before you decide - and the bootloader reverts. **Validity must be actively claimed.**

Trace the two failing paths, because the asymmetry is the design.

Class A fails and you roll back immediately. A missing sensor or an unusable storage partition is not going to fix itself in thirty seconds. Waiting only delays the inevitable while the device sits in an unknown state.

Class B fails and you retry first. A server that happens to be restarting is not evidence of a bad image. Rolling back over that is its own availability bug: you reverted good firmware because someone rebooted a laptop.

And note the branch that is easy to miss entirely — the one out the right-hand side. **Most boots are not probation boots.** After an image has been marked valid once, this whole flow is skipped, so the cost of all this machinery is paid exactly once per update rather than on every start.

NOTE**But do not roll back over a brief outage**

The opposite failure is just as real: rolling back a good image because the server happened to be restarting creates its own availability problem. Separate your checks into two classes.

Class	Examples	On failure
Boot-critical, local	NVS mounts, sensor answers, memory available	Roll back immediately. These cannot fix themselves.
Network reachability	update server responds	Retry with backoff over a <i>documented</i> budget — here, 3 attempts across about 18 s — and only then roll back.

15B.5 Calling it at the right moment

```
void app_main(void) {
    ESP_ERROR_CHECK(nvs_flash_init());
    ESP_ERROR_CHECK(esp_netif_init());
    ESP_ERROR_CHECK(esp_event_loop_create_default());
    ESP_ERROR_CHECK(example_connect());           // network up BEFORE the self-test runs
    i2c_bringup();                               // sensor up too

    const esp_partition_t *running = esp_ota_get_running_partition();
    esp_ota_img_states_t st;
    if (esp_ota_get_state_partition(running, &st) == ESP_OK &&
        st == ESP_OTA_IMG_PENDING_VERIFY) {
        ESP_LOGW("ota", "on probation -- running self-test");
        if (self_test()) {
            ESP_ERROR_CHECK(esp_ota_mark_app_valid_cancel_rollback());
            ESP_LOGI("ota", "image marked VALID");
        } else {
            ESP_LOGE("ota", "self-test FAILED -- rolling back");
            esp_ota_mark_app_invalid_rollback_and_reboot(); // does not return
        }
    }
    // ... normal application from here ...
}
```

The ordering is the point. The self-test checks the network and the sensor, so both must be brought up *before* it runs. Putting `confirm_or_rollback()` at the very top of `app_main`, as it is natural to do, would fail check 3 on every single boot and roll back a perfectly good image.

15B.5b The whole file, in one piece

§15B.4 gives each helper with its reasoning. This is the same code assembled into one `main.c` you can paste and build, so you are never hunting fragments across three sections.

```
// =====
// Lab 15B -- OTA with rollback actually enabled. COMPLETE FILE.
//
// Goal:   Update over the air, run a self-test on the new image, and prove
//         a bad image reverts itself without anyone present.
// Verify: Case 3 (self-test fails) and case 4 (reset during probation) BOTH
//         reboot into the previous slot.
// Gotcha: CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE is OFF by default. Without
//         it PENDING_VERIFY is never used and none of this does anything.
// Check:  Select-String build\config\sdkconfig.h -Pattern ROLLBACK
// =====
#include <stdio.h>
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_system.h"
#include "esp_ota_ops.h"
#include "esp_https_ota.h"
#include "esp_http_client.h"
#include "esp_wifi.h"
#include "esp_netif.h"
#include "esp_event.h"
#include "driver/i2c_master.h"
#include "nvs_flash.h"
#include "nvs.h"
#include "esp_log.h"
#include "protocol_examples_common.h" // needs EXTRA_COMPONENT_DIRS, see 27.1

#define HEALTH_URL "http://192.168.1.50:8070/health"
#define FIRMWARE_URL "http://192.168.1.50:8070/firmware.bin"

static const char *TAG = "lab15b";
static i2c_master_dev_handle_t s_imu;

// ----- I2C
static esp_err_t i2c_bringup(void)
{
    i2c_master_bus_config_t bc = {
        .i2c_port = I2C_NUM_0, .sda_io_num = GPIO_NUM_8, .scl_io_num = GPIO_NUM_9,
        .clk_source = I2C_CLK_SRC_DEFAULT, .glitch_ignore_cnt = 7,
        .flags.enable_internal_pullup = true,
    };
    i2c_master_bus_handle_t bus;
    esp_err_t e = i2c_new_master_bus(&bc, &bus);
    if (e != ESP_OK) return e;
    i2c_device_config_t dc = {
        .dev_addr_length = I2C_ADDR_BIT_LEN_7,
        .device_address = 0x68, .scl_speed_hz = 400000,
    };
    return i2c_master_bus_add_device(bus, &dc, &s_imu);
}

static esp_err_t mpu_read(uint8_t reg, uint8_t *dst, size_t n)
{
    return i2c_master_transmit_receive(s_imu, &reg, 1, dst, n, pdMS_TO_TICKS(100));
}

// ----- reachability probe
```

```

static bool health_check_ok(const char *url, int attempts, int timeout_ms)
{
    for (int i = 0; i < attempts; i++) {
        esp_http_client_config_t cfg = {
            .url = url, .method = HTTP_METHOD_GET, .timeout_ms = timeout_ms,
        };
        esp_http_client_handle_t c = esp_http_client_init(&cfg);
        if (!c) return false;

        esp_err_t err = esp_http_client_perform(c);
        int status = (err == ESP_OK) ? esp_http_client_get_status_code(c) : -1;
        esp_http_client_cleanup(c); // on BOTH paths, or you leak

        if (err == ESP_OK && status >= 200 && status < 300) {
            ESP_LOGI(TAG, "health ok (HTTP %d, attempt %d)", status, i + 1);
            return true;
        }
        ESP_LOGW(TAG, "health attempt %d/%d failed (%s, status %d)",
            i + 1, attempts, esp_err_to_name(err), status);
        vTaskDelay(pdMS_TO_TICKS(3000));
    }
    return false;
}

// ----- self-test
static bool self_test(void)
{
    // CLASS A -- boot-critical. Failure here rolls back immediately.
    nvs_handle_t h;
    esp_err_t e = nvs_open("storage", NVS_READONLY, &h);
    if (e == ESP_OK) {
        nvs_close(h);
    } else if (e == ESP_ERR_NVS_NOT_FOUND) {
        ESP_LOGI(TAG, "no 'storage' namespace yet (fresh unit) -- OK");
    } else {
        ESP_LOGE(TAG, "NVS unusable: %s", esp_err_to_name(e));
        return false;
    }

    uint8_t who = 0;
    if (mpu_read(0x75, &who, 1) != ESP_OK) {
        ESP_LOGE(TAG, "IMU did not ACK"); return false;
    }
    if (who != 0x70 && who != 0x68 && who != 0x71) {
        ESP_LOGE(TAG, "IMU WHO_AM_I = 0x%02X", who); return false;
    }

    // CLASS B -- reachability. Retried over a documented budget first.
    wifi_ap_record_t ap;
    if (esp_wifi_sta_get_ap_info(&ap) != ESP_OK) {
        ESP_LOGE(TAG, "not associated"); return false;
    }
    if (!health_check_ok(HEALTH_URL, 3, 3000)) {
        ESP_LOGE(TAG, "update server unreachable"); return false;
    }

    ESP_LOGI(TAG, "self-test PASS (rssi %d)", ap.rssi);
    return true;
}

```

```
// ----- OTA
static void do_ota(void)
{
    ESP_LOGI(TAG, "starting OTA from %s", FIRMWARE_URL);
    esp_http_client_config_t http = {
        .url = FIRMWARE_URL,
        .timeout_ms = 20000,
        .keep_alive_enable = true,
        // Path A (plain HTTP) needs "Allow HTTP for OTA" in menuconfig.
        // Path B: serve over TLS and add .crt_bundle_attach here instead.
    };
    esp_https_ota_config_t ota = { .http_config = &http };

    esp_err_t err = esp_https_ota(&ota);
    if (err == ESP_OK) {
        ESP_LOGW(TAG, "OTA OK -- rebooting into the new slot");
        vTaskDelay(pdMS_TO_TICKS(500));
        esp_restart();
    } else {
        ESP_LOGE(TAG, "OTA failed: %s -- staying on the current image",
            esp_err_to_name(err));
    }
}

// ----- app_main
void app_main(void)
{
    esp_err_t e = nvs_flash_init();
    if (e == ESP_ERR_NVS_NO_FREE_PAGES || e == ESP_ERR_NVS_NEW_VERSION_FOUND) {
        ESP_ERROR_CHECK(nvs_flash_erase());
        ESP_ERROR_CHECK(nvs_flash_init());
    }
    ESP_ERROR_CHECK(esp_netif_init());
    ESP_ERROR_CHECK(esp_event_loop_create_default());
    ESP_ERROR_CHECK(example_connect()); // network up BEFORE the self-test
    ESP_ERROR_CHECK(i2c_bringup());    // sensor up too

    const esp_partition_t *running = esp_ota_get_running_partition();
    ESP_LOGI(TAG, "running from partition '%s'", running->label);

    esp_ota_img_states_t st;
    if (esp_ota_get_state_partition(running, &st) == ESP_OK &&
        st == ESP_OTA_IMG_PENDING_VERIFY) {
        ESP_LOGW(TAG, "on probation -- running self-test");
        if (self_test()) {
            ESP_ERROR_CHECK(esp_ota_mark_app_valid_cancel_rollback());
            ESP_LOGI(TAG, "image marked VALID -- permanent");
        } else {
            ESP_LOGE(TAG, "self-test FAILED -- rolling back");
            esp_ota_mark_app_invalid_rollback_and_reboot(); // no return
        }
    } else {
        ESP_LOGI(TAG, "not on probation -- normal boot");
    }

    // Trigger an update 10 s after boot so you can watch the whole cycle.
    // A real product would do this on a schedule or on a command.
    vTaskDelay(pdMS_TO_TICKS(10000));
}
```

```
do_ota();

for (;;) {
    ESP_LOGI(TAG, "application running");
    vTaskDelay(pdMS_TO_TICKS(5000));
}
}
```

WARNING**Before you build this**

Set the two URLs to your laptop's address on the same network. Find it with `ipconfig` — on an iPhone hotspot it will look like `172.20.10.3`.

Add `EXTRA_COMPONENT_DIRS` to your top-level `CMakeLists.txt` or `example_connect()` will not exist and the Kconfig menu for Wi-Fi credentials will not appear. See §27.1 — this is the single most common reason a network lab stalls before it starts.

Confirm rollback is armed before trusting any result:

```
c Select-String -Path build\config\sdkconfig.h -Pattern "BOOTLOADER_APP_ROLLBACK"
```

If that prints nothing, cases 3 and 4 will silently pass and prove nothing.

15B.6 The four cases you must record

#	How to produce it	Expected
1	Serve a good image, trigger OTA	downloads, reboots, new slot runs, log shows "marked VALID"
2	Confirm which slot is running before and after	<code>ota_0</code> and <code>ota_1</code> alternate across updates
3	Build an image whose <code>self_test()</code> returns <code>false</code> , serve it	boots once, logs the failure, reboots into the previous slot
4	Build an image that calls <code>esp_restart()</code> before confirming	bootloader sees it still PENDING, rolls back with no cooperation from the firmware

Case 4 is the strongest evidence, because it proves the guarantee holds when the new firmware is too broken to participate. If your image crashes on boot instead, that works equally well as a demonstration.

KEY POINT**If case 3 and 4 do not roll back**

Go back to §15B.2 and `grep sdkconfig.h`. An un-enabled `CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE` produces exactly this symptom: cases 1 and 2 pass, 3 and 4 silently do not.

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 15.

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05	I ² C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15 ←	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.